A PROJECT REPORT
ON

"VITAMIN DEFFICIENCY BASED DISEASE PREDICTION AND FOOD RECOMMENDATION SYSTEM USING MACHINE LEARNING"

A project report submitted in the partial fulfillment of the requirements for the Award of the Degree of

Bachelor of Science

In

Computer Science and Artificial Intelligence

By

KATIGHAR ANJALI **(111722038003)**
SHAIK ANSHARUL HUSSAIN **(111722038035)**

DEPARTMENT OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

LOYOLA DEGREE AND PG COLLEGE

Old Alwal, Secunderabad

(Autonomous and affiliated to Osmania University)
Re-accredited by "NAAC" with "A" Grade
"A College with Potential for Excellence" by UGC.
Secunderabad

2025

LOYOLA ACADEMY
Old Alwal, Secunderabad – 500010
(An Autonomous Degree College affiliated to Osmania University)
Accredited by "NAAC" with "A" Grade
A "College with Potential for Excellence" by UGC.

Department of Computer Science and Artificial Intelligence

CERTIFICATE

This is to certify that the project report entitled for **"**is a record of Bonafide work carried out during 3rd Year, 6th Semester as Partial fulfilment for the award of **DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE** during the academic year 2024-25.

The results embodied in this project report have not been submitted to any other University of any Degree or Diploma.

NAME: KATIGHAR ANJALI & SHAIK ANSHARUL HUSSAIN
CLASS: DCSAI
UID: 111722038003 & 111722038035

HEAD OF THE DEPARTMENT PRINCIPAL

EXTERNAL EXAMINER INTERNAL EXAMINER

Myra IT Technologies

Certificate of Accomplishment

This is to certify

Katighar Anjali	**111722038027**
Shaik Ansharul Hussain	**111722038035**

LOYOLA ACADEMY (BSC Computer Science and Artificial Intelligence)

Are approved with facilities to do a project wok at

MYRA IT TECHNOLOGIES, Hyderabad

As per the following details

Proposed Project Titled

"VITAMIN DEFFICIENCY BASED DISEASE PREDICTION AND FOOD RECOMMENDATION SYSTEM USING MACHINE LEARNING"

This permission to do a project work was accorded on the request form the institution where students are studying as partial fulfillment of completion of the project.

The student has completed the project work and during the training period **December 2024 to April 2025**. The students evinced keen interest in the training. The conduct was good during the above period.

Yours Sincerely,
From Myra IT Technologies.

Kasarla shanthan
Managing Director.

Myra IT Technologies

518, Annapurna Block, Adithya Enclave, Ameerpet, Hyderabad, Telangana – 500016.India Mobile: 7702177291

Myra IT Technologies

CERTIFICATE

This is to certify that **Katighar Anjali (111722038003) and Shaik Ansharul Hussain (111722038035)** From LOYOLA ACADEMY DEGREE AND PG COLLEGE, OLD ALWAL, SECUNDERABAD, 500010, have successfully done a "COLLABRATIVE RESEARCH" work with Myra IT Technologies on the project entitled,
"**AI-Powered Fashion Finder: Personalized Clothing Recommendations by Body** "

They have done this work under the guidance and supervision of **MRS.S. SARASHRI (Assistant Professor)** from the department of B. Sc. Computer Science and Artificial Intelligence LOYOLA ACADEMY, OLD ALWAL, SECUNDERABAD 500010. This work was done during the period **December 2024 to April 2025**

Yours Sincerely,
From Myra IT Technologies.

Kasarla shanthan
Managing Director.

Myra IT Technologies

518, Annapurna Block, Adithya Enclave, Ameerpet, Hyderabad, Telangana – 500016.India Mobile: 7702177291

Myra IT Technologies

CERTIFICATE

This is to certify that

Katighar Anjali (111722038003) and Shaik Ansharul Hussain (111722038035)

from LOYOLA ACADEMY DEGREE AND PG COLLEGE, OLD ALWAL, SECUNDERABAD, 500010, successfully has a MOU to do "COLLABRATIVE RESEARCH "work with Myra IT Technologies on the project entitled

"VITAMIN DEFFICIENCY BASED DISEASE PREDICTION AND FOOD RECOMMENDATION SYSTEM USING MACHINE LEARNING"

They have done this work under the guidance and supervision of **MRS.S. SARASHRI (Assistant Professor)** from the department of B. Sc. Computer Science and Artificial Intelligence LOYOLA ACADEMY, OLD ALWAL, SECUNDERABAD 500010. This work was done during the period **December 2024 to April 2025**

Yours Sincerely,
From Myra IT Technologies.

Kasarla shanthan
Managing Director.

Myra IT Technologies

518, Annapurna Block, Adithya Enclave, Ameerpet, Hyderabad, Telangana – 500016.India Mobile: 7702177291

DECLARATION

We hereby declare that the project entitled **"VITAMIN DEFFICIENCY BASED DISEASE PREDICTION AND FOOD RECOMMENDATION SYSTEM USING MACHINE LEARNING"** done by **KATIGHAR ANJALI** (111722038003) & **SHAIK ANSHARUL HUSSAIN** (111722038035) submitted to, **Mrs. S.Sarashri** Assistant Professor in B.Sc Computer Science and Artificial Intelligence**,** Loyola Academy Degree and PG College is a record of original work done by us.

The project has been successfully completed and submitted in partial fulfilment of the requirements for the award of "**DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE**" from "Loyola Academy Degree and PG College", affiliated to "Osmania University, Hyderabad" in an authentic way and has not been submitted in any other university or institution for the award of degree or diploma.

KATIGHAR ANJALI **SHAIK ANSHARUL HUSSAIN**

(111722038003) **(111722038035)**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of this major project would be in complete without the mention of the people who made it possible, without whose constant guidance and encouragement would have made efforts go in vain. I consider myself privileged to express gratitude and respect towards all those who guided us through the completion of this project.

On successful completion I convey thanks to my project guide **Mrs.S.Sarashri, Assistant Professor** of the Department of Computer science and Artificial Intelligence for providing encouragement, constant support and guidance which was of great help to complete this project successfully.

I am grateful to Mrs. Bala Seshu Kumari, Head of the Department of for giving me the support and encouragement that was necessary for the completion of this project.

VITAMIN DEFICENCY BASED DISEASE PREDICTION AND FOOD RECOMMENDATION

ABSTRACT

According to a World Health Organization (WHO) study, poor and unbalanced nutrition is responsible for approximately 9% of heart attack fatalities, around 11% of deaths due to ischemic heart disease, and about 14% of global gastrointestinal cancer deaths. Additionally, approximately 250 million children suffer from various vitamin deficiencies—from Vitamin A to Vitamin K. Around 200 million people are affected by iron deficiency (anemia), while about 700 million face iodine deficiency.

The primary goal of this project is to develop a personalized dietary recommendation system. The system is designed to process large volumes of data and suggest appropriate food intake based on an individual's vitamin levels.

A custom dataset was created using varied levels of vitamins (A, B, C, D, E, and K), categorized into normal and abnormal ranges. These conditions are labeled as '0' for normal and '1' for abnormal. A second dataset links specific vitamin deficiencies to suitable food recommendations. A third dataset maps vitamin deficiencies to related health conditions, allowing disease prediction.

To build the model, several classification algorithms were implemented, including K-Nearest Neighbours (KNN), Decision Tree, Random Forest, Logistic Regression, and a Voting Classifier. An ensemble approach was used to combine the strengths of multiple models and enhance overall performance. Each model’s accuracy was evaluated, and the most accurate model was selected for making final predictions.

The results are integrated into a Flask-based web application, which identifies vitamin deficiencies, predicts potential diseases associated with those deficiencies, and recommends appropriate foods based on the detected nutrient gaps.

TABLE OF CONTENTS

CHAPTERS	TITLE	Page.No
CHAPTER-1	1.1 Introduction	1
CHAPTER-2	2.1 Literature Review	5
CHAPTER-3	3.1 Introduction	8
	3.2 Introduction to Python	11
CHAPTER-4	4.1 Existing System	27
	4.1.1 Disadvantages of Existing System	27
	4.2 Proposed System	28
	4.2.1 Advantages of Proposed System	28
CHAPTER-5	5.1 Introduction	29
	5.2 Modules	35
	5.2.1 Dataset	26
	5.2.2 Pre-processing	26
	5.2.3 Methodology	29
	5.3 System Architecture	31
	5.4 UML Diagrams	34
	5.4.1 Use Case Diagrams	39

CHAPTERS	TITLE	Page.No
	5.4.2 Sequence Diagrams	39
	5.4.3 Class Diagram	41
	5.4.4 Activity Diagram	41
CHAPTER-6	6.1 System Requirements	42
CHAPTER-7	7.1 Input and Output Designs	43
	7.1.1 Logical Design	43
	7.2.2 Objectives	45
	7.2.3 Output Design	45
CHAPTER-8	8.1 Introduction	51
	8.2 Levels of Testing	51
	8.2.1 Black Box Testing	52
	8.2.2 White Box Testing	54
CHAPTER-9	9.1 Main Page	61
CONCLUSION		64
REFERENCES		65

CHAPTER-1
INTRODUCTION

1.1 INTRODUCTION

In today's world, people are increasingly facing a variety of health challenges, including issues related to physical fitness, poor dietary habits, and mental well-being. Numerous studies indicate that one of the primary contributors to these health problems is an imbalanced and insufficient diet. A study by WHO reports that inadequate and imbalanced intake of food causes around 9% of heart attack deaths, about 11% of ischemic heart disease deaths, and 14% of gastrointestinal cancer deaths worldwide. Additionally, approximately 250 million children suffer from Vitamin A deficiency, around 200 million individuals are affected by iron deficiency (anemia), and nearly 700 million people face iodine deficiency worldwide. The primary aim of this project is to recommend personalized dietary plans to individuals based on their specific needs. The recommender system is designed to process a large amount of data, filtering out the most relevant information based on user input and preferences. It identifies suitable matches between users and food items by analyzing both physical characteristics—such as age, gender, height, weight, and body fat percentage—and personal goals, like weight loss or weight gain.
The recommendation process is divided into three main stages: the Information Collection Phase, the Learning Phase, and the Recommendation Phase. In the first stage, relevant user information and possible dietary solutions are gathered and categorized. The second stage involves analyzing the collected data to derive meaningful patterns and insights. Finally, in the Recommendation Phase, personalized food suggestions are generated based on the user's physical attributes, preferences, and Body Mass Index (BMI).

Balanced nutrition is a key component of a healthy lifestyle. Along with a well-rounded diet, regular physical activity plays a vital role in maintaining overall health. However, in today's fast-paced world, nutrition and health are often neglected. A large portion of the population suffers from chronic conditions such as diabetes, heart disease, cancer, and stroke—many of which are directly linked to poor dietary habits.

Our bodies require essential nutrients to function properly and to protect us from illness. A healthy, balanced diet typically consists of a variety of vitamins, minerals, proteins, healthy fats, carbohydrates, and dietary fiber. The food pyramid emphasizes the importance of plant-based foods, moderate consumption of animal products, and a limited intake of oils and sweets. It generally includes vegetables, grains, fruits, dairy, meat, and legumes.

Many individuals are unaware of the root causes behind nutrient deficiencies or excesses—such as those related to calcium, protein, or vitamins—and how these imbalances can be addressed through proper diet. With advancements in technology, it is now possible to support individuals in adopting healthier lifestyles through intelligent systems.

This project focuses on developing a system that recommends personalized nutritional intake based on a user's Body Mass Index (BMI) and their grocery preferences. BMI helps classify individuals into categories such as underweight, healthy weight, overweight, or obese. The grocery data used in this system includes seasonal foods, user-preferred items, plant-based options, and animal products, allowing for personalized and practical dietary recommendations.

Algorithm

KNN Classifier Algorithm:

The K-Nearest Neighbors (KNN) algorithm is a versatile method that can be applied to both regression and classification tasks in predictive modeling. This method helps in interpret output, calculate time and predictive power. The Machine learning techniques are used in various fields. KNN is also one of the machine learning method. This is also called as method of sample-based learning. This approach utilizes historical datasets, which can be used to identify patterns and make predictions for new, unseen data to use while predicting the new datasets. This will apply function called as distance function like Manhattan or Euclidean distance. This can be used to compute distance from samples to all other training samples. It calculates the target value for new samples. The target value is determined by the weighted sum of the target values of the *k* nearest neighbors. The value of *k* is directly related to prediction behavior. A small *k* value leads to high variance and low bias, while a large *k* results in low variance and high bias. A key benefit of KNN is that it requires no training or optimization phase. Instead, it relies on existing data samples to make predictions on new datasets. As a result, the algorithm has higher complexity and takes more time to compute.

This work represents a review of K-NN technique for the early prediction of food recommendation. K-NN analysis is used for predicting the unknown parameter from the known parameters. In this work we are considering vitamins as input parameters which are the main parameters to be considered for a good food recommendation, although there are many other factors that can be considered. The unknown value of vitamins can be predicted from the nearest known values of the nearest neighbors by calculation of Euclidean distance between them. Then we would be able to predict type of food for given vitamin parameters. To measure the distance between points in a feature space, various distance functions can be used, in which the Euclidean distance function is the most widely used one. Let p and q are represented as feature vectors. To calculate the distance between p and q, the Euclidean metric is generally used by if a= (a1, a2) and b=(b1, b2) then the distance is given

Classifications Algorithms

The next crucial phase involves training the machine learning algorithms. To be able to test the performance of our algorithms, I first performed an 80/20 train-test split, splitting our balanced data set into two pieces. To avoidoverfitting, I used the very common resampling technique of k-fold cross-validation. This simply means that you separate your training data into k parts (folds) and then fit your model on k-1 folds before making predictions for the kth hold-out fold. You then repeat this process for every single fold and average the resulting predictions.

To get a better feeling of which algorithm would perform best on our data, let's quickly spot-check some of the most popular classification algorithms:

- Logistic Regression
- Linear Discriminant Analysis
- K Nearest Neighbors (KNN)
- Classification Trees
- Support Vector Classifier
- Random Forest Classifier

Logistic regression: This is a classification algorithm that builds a predictive model using a single multinomial logistic regression estimator. Logistic regression defines a decision boundary between classes and estimates class probabilities based on the distance from this boundary. As the dataset grows, these probabilities tend to approach the extremes (0 or 1) more quickly. These probability estimates make logistic regression more than just a simple classifier; it provides detailed predictions. However, these confident predictions can sometimes be incorrect. Similar to Ordinary Least Squares (OLS) regression, logistic regression is a method for making predictions, but its outcomes are binary in nature. It remains one of the most widely used methods in applied statistics and discrete data analysis. Essentially, logistic regression can be viewed as a form of linear interpolation tailored for classification tasks.

CHAPTER- 2
LITERATURE SURVEY

2.1 LITERATURE REVIEW

Raciel Yera Toledo proposed a food recommender system that incorporates both nutritional data and user preferences. The meal plans in the system are generated by considering individual dietary choices alongside nutritional needs, ensuring a balanced recommendation.

Vijay Jaiswal developed a system focused on promoting healthy food habits and tracking calorie intake using data mining techniques. By analysing hidden patterns and consumer eating behaviours from various data sources, algorithms such as Decision Tree and Random Tree were applied to classify and recommend nutritional intakes.

H. Jiang introduced a system for calculating daily calorie requirements, employing the Knapsack algorithm to recommend optimal diet combinations. Unlike conventional diabetic diet systems, this approach ranks recommended diets using the TOPSIS algorithm based on nutritional values relevant to the user.

Jung-Hyun Lee designed a personalized diet recommendation service tailored for individuals with heart disease. The system customizes suggestions based on general health data, family medical history, and seasonal food consumption.

Rung-Ching Chen built a recipe ontology to associate certain foods with healing properties for common diseases. The system uses an inference engine to evaluate customer health conditions and provide recipe recommendations based on food priorities.

Fidelson Tanzil applied the Artificial Bee Colony (ABC) algorithm to extract data according to user requirements. Clustering algorithms like K-Means and Self-Organizing Maps (SOM) were used to analyze food consumption patterns. Mohd Afisi also utilized the ABC algorithm in data mining and demonstrated its effectiveness in classification tasks when compared with six traditional methods.

Xiaoyan Gao addressed food recommendation challenges by considering user preferences in recipe suggestions. A neural network-based approach was proposed to offer ordered diet recommendations based on these preferences.

Ingmar Weber and Palakorn Achananuparp explored diet success prediction through machine learning. They analyzed public food diaries of over 4,000 MyFitnessPal users to identify patterns of success or failure in meeting calorie goals. Their study leveraged a machine-learned model, finding that token-based representations outperformed category-based ones. Their work focused on "quantified self" data to better understand user behaviour.

Nandish Shah and Ishani Shah proposed a web-based food habit tracking system using data mining to discover patterns in eating behaviour. By analyzing customer data, the system recommended healthy diets while discouraging foods that contribute to illness. The authors used algorithms like classification, clustering, and association rule mining to extract insights and assess the nutritional structure of recipes, ultimately offering personalized health recommendations.

Aine P. Hearty and Michael J. Gibney demonstrated the use of data mining at the meal level, evaluating dietary quality through supervised learning techniques. Using data from the North-South Ireland Food Consumption Survey (1997–1999), they developed a Healthy Eating Index (HEI) and applied Artificial Neural Networks (ANNs) and Decision Trees to predict HEI quintiles. While ANNs showed slightly higher accuracy overall, Decision Trees performed better when used with meal-based coding.

Finally, Christy Samuel Raju and team created a desktop application, **Fitness Advisor**, aimed at diagnosing weight-related issues and spreading awareness of health risks. The system considered multiple personal factors (e.g., BMI, habits, activity levels) and combined clustering, association, and classification techniques. The Apriori algorithm was used to generate association rules, and the system ultimately provided expert dietary and fitness advice.

CHAPTER-3
THEORETICAL BACKGROUND

3.1 INTRODUCTION:

What is Machine Learning?

Tom Mitchell defined machine learning as: "A computer program is said to learn from experience *E* with respect to some task *T* and performance measure *P*, if its performance on *T*, as measured by *P*, improves with experience *E*." In essence, machine learning is the process of uncovering patterns and relationships in data. Most machine learning algorithms focus on identifying these connections between datasets to either predict future outcomes or extract meaningful patterns from complex data.

Machine learning includes a wide array of algorithms such as Regression, Linear Regression, Logistic Regression, Naive Bayes, Bayes' Theorem, K-Nearest Neighbors (KNN), Decision Trees, Entropy-based algorithms like ID3, Support Vector Machines (SVM), K-Means Clustering, and Random Forests, among others.

The term *machine learning* was first introduced by Arthur Samuel in 1959. It refers to the study and development of algorithms that learn from data and make data-driven predictions or decisions. Closely related to fields like computational statistics, machine learning often overlaps with predictive modeling and mathematical optimization, providing powerful tools and techniques to enhance data analysis and problem-solving. Though sometimes confused with data mining, machine learning focuses more on building predictive models, whereas data mining emphasizes pattern discovery, especially in the context of unsupervised learning.

In data analytics, machine learning is widely used to develop sophisticated models capable of predictive analysis. These models support data scientists, engineers, and analysts in making consistent, accurate decisions by learning from past data and trends to reveal hidden insights.

Machine learning tasks are commonly categorized into several main types:

- Supervised Learning: The model learns from a labeled dataset, where each input is paired with the correct output, to infer a mapping function that can predict outcomes for new data.

- Semi-Supervised Learning: The model is trained on a dataset where only a portion of the data is labelled, making it more efficient in cases where labelling is costly or time-consuming.
- Active Learning: The algorithm actively selects the most informative data points from which it wants labels, typically under a limited budget or resource constraint.
- Unsupervised Learning: The model works with unlabeled data to uncover hidden structures or groupings in the dataset, often used for clustering and dimensionality reduction.
- Reinforcement Learning: The model learns by interacting with an environment, receiving feedback in the form of rewards or penalties based on its actions, as seen in robotics, gaming, and autonomous systems.

Benefits of Machine Learning:

1. **Achieve economies of scale** – By increasing output or productivity with fewer personnel, organizations can significantly reduce their cost per unit, project, or product—resulting in greater efficiency and profitability.
2. **Reduce spending on technology infrastructure.** Maintain easy access to your information with minimal upfront spending. Pay as you go (weekly, quarterly or yearly), based on demand.
3. **Globalize your workforce on the cheap.** People worldwide can access the cloud, provided they have an Internet connection.
4. **Streamline processes.** Get more work done in less time with less people.
5. **Reduce capital costs.** Significant expenditure on hardware, software, or licensing fees is unnecessary, as cost-efficient alternatives can achieve comparable outcomes.
6. **Improve accessibility.** You have access anytime, anywhere, making your life so much easier.
7. **Monitor projects more effectively.** Stay within budget and ahead of completion cycle times.
8. **Less personnel training is needed.** It takes fewer people to do more work on a cloud, with a minimal learning curve on hardware and software issues.

9. **Minimize licensing new software.** Stretch and grow without the need to buy expensive software licenses or programs.
10. **Improve flexibility.** You can change direction without serious “people” or “financial” issues at stake.

Advantages:

- **Cost-Efficiency**: Pay only for the resources used, optimizing expenses**.**
- **Security:** Network isolation enhances protection against unauthorized access.
- **Performance:** Instantly scale instances to access full hardware resources.
- **Scalability:** Auto-deploy instances based on demand for seamless scaling.
- **Availability:** Redundant servers ensure uptime; workloads shift automatically on failure.
- **Remote Control:** Access and manage instances from anywhere with snapshots and software libraries.
- **Traffic Handling:** Quickly deploy additional instances to manage traffic spikes**.**

3.2 INTRODUCTION TO PYTHON

Python Technology

Python technology is both a programming language and a platform.

The python Programming Language

THE PYTHON PROGRAMMING LANGUAGE IS A HIGH-LEVEL LANGUAGE THAT CAN BE CHARACTERIZED BY ALL OF THE FOLLOWING BUZZWORDS:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust

- Dynamic
- Secure

In most programming languages, a program must be either compiled or interpreted before it can be executed on a computer. The Python programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called Python byte codes —the platform-independent codes interpreted by the interpreter on the Python platform. The interpreter parses and runs each Python byte code instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed. The following figure illustrates how this works.

FEATURES OF MACHINE LEARNING

- It is nothing but automating the Automation.
- Getting computers to program themselves.
- Writing Software is bottleneck.
- Machine leaning models involves machines learning from data without the help of humans or any kind of human intervention.
- Machine Learning is the science of making of making the computers learn and act like humans by feeding data and information without being explicitly programmed.
- Machine Learning is totally different from traditionally programming, here data and output is given to the computer and in return it gives us the program which provides solution to the various problems. Below is the figure.

Traditional Programming vs Machine Learning

- Machine Learning is a combination of Algorithms, Datasets, and Programs.

- There are Many Algorithms in Machine Learning through which we will provide us the exact solution in predicting the disease of the patients.

- How Does Machine Learning Works?

- Solution to the above question is Machine learning works by taking in data, finding relationships within that data and then giving the output.

Machine Learning Model

- There are various applications in which machine learning is implemented such as Web search, computing biology, finance, e-commerce, space exploration, robotics, social networks, debugging and much more.

- There are 3 types of machine learning supervised, unsupervised, and reinforcement.

BENEFITS OF PYTHON

- Presence of Third-Party Modules

- Extensive Support Libraries

- Open Source and Community Development

- Learning Ease and Support Available

- User-friendly Data Structures

- Productivity and Speed

- Highly Extensible and Easily Readable Language.

Python

Python is high level language and it is also integrated version of the program. Python is an object-oriented approach and its main aim to help programmers to write the code clearly, logical code for small and large scale of project.

Pytrhon is dynamically typed and garbage collected it also support multiple programming and it is both procedure and object oriented and also functional programming. And structural programming also supported. It has many built in function it also supports filter, map and reduce function. All the machine learning algorithm and the libraries are being supported by the python programming language. Python also support list, dict, sets and other generators. Python code can be run in different platform such as anaconda, PyCharm etc.

The main goal of this programing language is as follows:

- Python is simple, object-oriented programming language.
- The language and implementation should provide support for software engineering principles such as strong type library preset for different machine learning algorithm, and all other algorithm in simple manner.
- Coding will be smooth in python and the data analysis can be easily done in python.

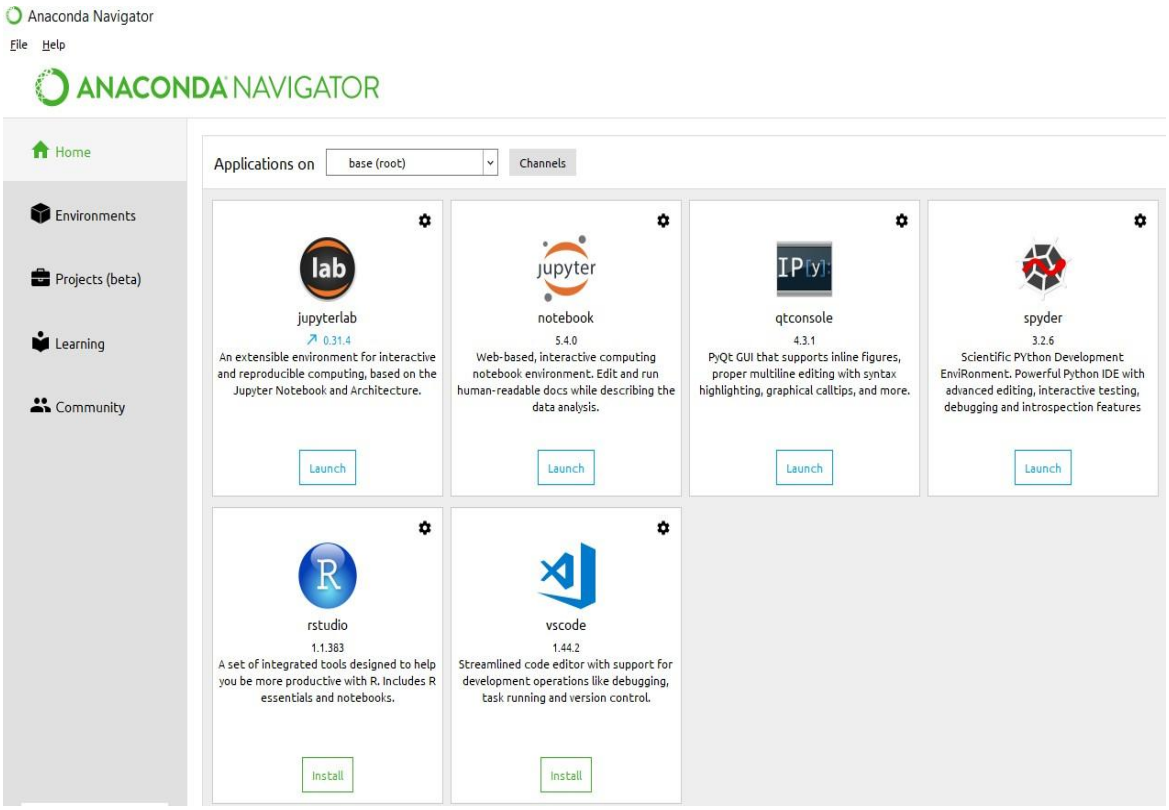
Machine learning has become remarkably accessible, to the point where one can utilize modules and APIs with minimal understanding of the underlying mechanics. Tools like Scikit-learn offer default settings that can achieve 90–95% accuracy on many tasks straight out of the box. Much like driving a car, one can leverage the power of machine learning without fully grasping its internal workings.

Despite its long history and growing maturity, there has never been a better time to learn machine learning. The tools are user-friendly, modern machines are highly capable, and the availability of data has never been greater—making it both practical and rewarding to get started today.

Anaconda

Anaconda is a free and open-source distribution of the Python and R programming languages, designed for scientific computing, including data science, machine learning, large-scale data processing, and predictive analytics. Developed and maintained by Anaconda, Inc., it simplifies package management and deployment across Windows, Linux, and macOS platforms.

Anaconda includes a wide range of data science packages, managed by its own package management system, **conda**. Initially part of Anaconda, **conda** was later released as a standalone open-source tool due to its broader applicability beyond Python. A minimal version of Anaconda, called **Miniconda**, is also available—it contains only **conda**, Python, and a few essential packages, allowing users to install only what they need.



Anaconda Console

Jupyter notebook

Jupiter Notebook or so called IPython Notebook is an interactive web based computational mean for starting with Jupiter Notebook documents. The term notebook itself is a huge entity to represent the integration with different entity sets. **JSON (JavaScript Object Notation)** is a widely used data-interchange format that defines a schema for input and output execution. It offers high compatibility with various programming languages and provides flexibility in terms of structure and data representation, making it ideal for integration across different systems. The extension used for the same is ". ipynb" which runs in this platform. It's an open-source software package with interactive communication means. It has its open standards for the same. It's an open community best for budding programmers. The flexibility of JSON is remarkable, allowing for seamless configuration and integration. Its simplicity ensures minimal disruption during implementation, and its efficiency can be consistently maintained across a wide range of systems.

This software suite is among the most widely adopted tools for designing and developing products, offering extensive support and scalability. It enables flexible coding and deployment, supports multiple programming languages, and allows projects to be undertaken seamlessly across platforms. Notebooks created within this environment can be stored and shared through various means, facilitating collaboration and future use.

It supports interactive and well-structured outputs, making it ideal for graphing, plotting, and data visualization. The platform excels in data integration and is capable of handling large volumes of data efficiently, resulting in improved performance and higher computational capacity. Key data-related tasks such as data cleaning, transformation, modeling, and visualization can all be effectively executed using this tool.

Machine learning, in this context, refers to a computer's ability to learn from data and improve its performance over time without being explicitly programmed. There are two types of machine learning:

Supervised Learning: Supervised learning is the learning of the labelled data. It is the types of machine learning that maps the input and output based on the examples input-output pairs. In

supervised learning each training data having pairs of input and desired outputs values. Supervised learning algorithm analyzes the training data and produces a function which can be used for mapping of new data.

Fig 2.1 Supervised Learning The output to solve the supervised learning algorithm are as:

- Determine the types of data, before doing anything else the user should understand which types of data set is to be used for training the data.
- Gathered the training data sets either in form of human experts or from measurements.
- Determine the feature of inputs from the learned data and depends on the inputs it changed into feature vector; number of features should not be large but should contains enough information to accurately predict the outputs.
- Check the learned function and the learned algorithm for example we use support vector machines or decisions tree.
- Complete the design and run the trained data sets.
- Analyzed the output and verify the data sets to get the accurate outputs.

Unsupervised Learning:

Unsupervised learning is a type of machine learning that helps in finding the previously unknown patterns in the data set without any known labels. It is known as self- organization and allows modelling probability densities of given inputs.

Fig 2.2: Unsupervised Learning – The following algorithms are commonly applied in unsupervised learning:

- Clustering
- Anomaly detection
- Neural networks
- Approach for learning latent variable models
- Non labelled data

Semi Supervised Machine Learning algorithm: It's like the middle man which have some labeled data and some unlabeled which can be prosed by the both the structured and unsupervised learning.

The algorithms are evaluated based on parameters such as dataset size and the number of technical indicators utilized. For each algorithm, accuracy and F-measure values have been calculated. A long-term model was employed to derive these performance metrics.

Reinforcement Learning: This learning approach aims to strengthen the network through feedback received from the environment, referred to as "critic information." Unlike supervised learning, where feedback is instructive, reinforcement learning provides evaluative feedback, which guides the network to adjust its weights to improve future performance. Although similar to supervised learning, reinforcement learning often operates with limited information. The block diagram below illustrates the reinforcement learning process.

import numpy as np

1. **NumPy** is a core Python library for scientific computing. It offers a powerful N-dimensional array object (ndarray), along with tools for performing fast numerical operations such as mathematical functions, logical operations, shape manipulation, sorting, selection, I/O, linear algebra, statistics, Fourier transforms, and random simulations.
2. At the heart of NumPy is the ndarray, which allows for high-performance operations through compiled code. There are key differences between NumPy arrays and standard Python sequences:

   - NumPy arrays have a fixed size once created, unlike Python lists which are dynamic. Resizing requires creating a new array.

   - All elements in a NumPy array must be of the same data type, ensuring memory consistency. Arrays of Python objects are an exception and can hold varied types.

   - NumPy enables efficient computation on large datasets with less code than Python's built-in data structures.

3. Many scientific and mathematical libraries built on Python use NumPy arrays internally. Although they may accept standard Python sequences as input, these are typically converted to NumPy arrays for processing. Therefore, understanding NumPy is essential for working effectively with modern scientific Python ecosystems

import pandas as pd

- **Pandas** is a high-level data manipulation library created by **Wes McKinney**. Built on the **NumPy** package, its main data structure, the **DataFrame**, enables storing and handling tabular data as rows of observations and columns of variables.
- **Pandas** is a powerful data analysis library created by **Wes McKinney**. Built on **NumPy**, its primary data structure, the **DataFrame**, allows efficient storage and handling of tabular data with rows as observations and columns as variables.

import matplotlib

- **Matplotlib** is a powerful Python library for creating 2D plots from arrays. It's a cross-platform data visualization tool built on **NumPy** arrays and works seamlessly with the broader **SciPy** ecosystem. It was developed by **John Hunter** in 2002.
- A key advantage of visualization is providing visual access to large datasets in an intuitive form. **Matplotlib** supports various plots such as line, bar, scatter, histogram, and more.

import seaborn as sns

- **Seaborn** is a Python library used for creating statistical graphics. It is built on top of **Matplotlib** and works closely with **Pandas** data structures.
- **Seaborn** simplifies the process of exploring and understanding data. Its functions work on data frames or arrays, handling semantic mapping and statistical aggregation to create meaningful plots. Its declarative, dataset-oriented API emphasizes the meaning of plot elements over drawing mechanics.

import sklearn

- In general, a learning problem involves analyzing a set of *n* data samples to predict the properties of unknown data. If each sample contains multiple values—also known as multivariate data—it is considered to have multiple attributes or features.
- Learning problems are commonly categorized into:

  Supervised learning, where the data includes additional attributes to be predicted (Scikit-learn supervised learning). This can be:

- **Classification**: When samples belong to two or more categories, and the goal is to predict the class of new, unlabeled data. For example, in handwritten digit recognition, each input must be assigned to a specific class. Classification is a discrete type of supervised learning.

- **Regression**: When the target output is continuous. A common example is predicting the length of a salmon based on its age and weight.

Unsupervised learning, where the training data consists only of input vectors *x* without target labels. The goal might be to identify patterns, such as:

- **Clustering**: Grouping similar examples.
- **Density Estimation**: Understanding data distribution.
- **Dimensionality Reduction**: Projecting high-dimensional data into 2D or 3D for visualization (Scikit-learn unsupervised learning).

import imblearn

- In machine learning, while building a classification model we sometimes come to situations where we do not have an equal proportion of classes. That means when we have class imbalance issues for example, we have 500 records of 0 class and only 200 records of 1 class. This is called a class imbalance. All machine learning models are designed in such a way that they should attain maximum accuracy but in these types of situations, the model gets biased towards the majority class and will, at last, reflect on precision and recall. So how to build a model on these types of data set in a manner that the model should correctly classify the respective class and does not get biased.
- To get rid of these imbalance class issues few techniques are used called as Imblearn Technique that is mainly used in these types of situations. Imblearn techniques help to either up sample the minority class or down sample the majority class to match the equal proportion. Through this article, we will discuss imblearn techniques and how we can use them to do up sampling and down sampling. For this experiment, we are using Pima Indian Diabetes data since it is an imbalance class data set. The data is available on Kaggle for downloading.

import sklearn.metrics as m

There are three primary APIs for evaluating the quality of a model's predictions:

• **Estimator score method**: Each estimator has a score method that provides a default evaluation metric suited to its specific task. This method is explained in the documentation of each estimator rather than on this page.

• **Scoring parameter**: Tools that perform model evaluation using cross-validation - such as model_selection.cross_val_score and model_selection.GridSearchCV - rely on a scoring strategy defined by the scoring parameter. This is discussed in the section *The scoring parameter: defining model evaluation rules*.

• **Metric functions**: The metrics module offers a variety of functions for assessing prediction errors tailored to different tasks. These include Classification metrics, Multilabel ranking metrics, Regression metrics, and Clustering metrics.

Lastly, **Dummy estimators** serve as useful baselines, offering reference values for these metrics when using random or simplistic prediction strategies.

from sklearn.cluster import KMeans

- Clustering of unlabelled data can be performed with the module sklearn.cluster.
- Each clustering algorithm has two forms: a class that uses the fit method to learn clusters from training data, and a function that returns an array of integer labels for the clusters. For the class, these labels can be accessed using the labels_ attribute.

from imblearn.under_sampling import RandomUnderSampler

Imbalanced datasets occur when the number of instances in one class significantly outweighs those in another, often resulting in ratios like 1:100 or even 1:1000. This uneven distribution can

negatively impact the performance of many machine learning models, as they may overlook the underrepresented class—which is often the most critical for accurate predictions.

A common method to address this issue involves adjusting the dataset through random resampling. There are two primary techniques: **undersampling**, where samples from the majority class are removed, and **oversampling**, where instances from the minority class are replicated to balance the dataset.

In this guide, you will learn how random resampling can help improve model performance when dealing with imbalanced data. By the end, you'll understand that:

- Random resampling offers a basic but effective method for adjusting class imbalance.
- Oversampling involves repeating minority class samples, which may increase the risk of overfitting.
- Undersampling reduces the majority class size, which can lead to a loss of valuable information.

from sklearn.preprocessing import StandardScaler

- Standardization involves transforming features so they have a mean of zero and a standard deviation of one.
- The standardized value (z-score) of a sample x is computed as:

z = (x - u) / s

where u is the mean of the training data (or zero if with_mean=False), and s is the standard deviation (or one if with_std=False).

- Each feature is scaled independently by calculating its mean and standard deviation using only the training data. These values are then applied to transform future datasets.
- Many machine learning models require data to be standardized. If features vary widely in scale or are not centered, it can hinder the model's ability to learn effectively.
- For example, elements like the RBF kernel in SVMs or the L1/L2 penalties in linear models work best when features are on similar scales. If one feature has a much larger

variance than others, it could overpower the model and reduce its ability to learn from the rest of the input features.

from sklearn.model_selection import train_test_split

- Before exploring train_test_split, it's important to understand Scikit-learn (or Sklearn), a popular Python library that provides tools for data processing, classification, clustering, and model selection.
- The model_selection module is used to create a strategy for analyzing data and evaluating it on new, unseen data. Choosing the right model helps in making accurate predictions.
- This involves training a model using part of the dataset and then testing its performance using a separate portion.
- If you only have one dataset, it must be divided, which can be done using the train_test_split function from Sklearn.
- train_test_split helps you split datasets into training and testing sets automatically, eliminating the need to do it manually.
- By default, it partitions the data randomly, but you can control the randomness by setting a specific random_state value.

import seaborn as sns

- Seaborn is a powerful Python library built on top of Matplotlib, designed for creating visually appealing and informative statistical plots.
- To understand the library's core concepts, you can start with an introductory overview. The installation guide helps you set up Seaborn, while the example gallery showcases its capabilities. For deeper learning, the tutorial and API documentation provide comprehensive instructions.

- The library’s source code and issue tracker are available on its GitHub repository. For community support, Stack Overflow and Discourse offer helpful channels specifically for Seaborn-related queries

from sklearn.model_selection import cross_val_score

- Using the same dataset to both train and evaluate a prediction model is a flawed approach. A model that merely memorizes the training labels may achieve a perfect score on known data but perform poorly on new, unseen inputs—a problem known as **overfitting**.
- To prevent this, it is standard practice in supervised machine learning to separate a portion of the dataset as a test set (X_test, y_test).
- The term "experiment" here isn't limited to academic contexts; machine learning projects in industry also typically begin with experimentation.
- A common approach to model training involves **cross-validation**, which helps in evaluating performance more reliably.
- Optimal parameters for the model can be found using **grid search** techniques, ensuring better generalization.

import tensorflow as tf

To do machine learning TensorFlow, you are likely to need to define, save, and restore a model.

A model is, abstractly:

- A function that computes something on tensors (a forward pass)
- Some variables that can be updated in response to training

In this guide, you will go below the surface of Keras to see how TensorFlow models are defined. This looks at how TensorFlow collects variables and models, as well as how they are saved and restored.

from tensorflow.keras import layers, models, optimizers, callbacks, regularizers, metrics

When to use a Sequential model

A Sequential model is appropriate for a plain stack of layers where each layer has exactly one input tensor and one output tensor.

A Sequential model is not appropriate when:

- Your model has multiple inputs or multiple outputs
- Any of your layers has multiple inputs or multiple outputs
- You need to do layer sharing
- You want non-linear topology (e.g. a residual connection, a multi-branch model)

- ## from sklearn import metrics

Accuracy classification score.

In multilabel classification, this function computes subset accuracy: the set of labels predicted for a sample must exactly match the corresponding set of labels in y_true.

CHAPTER-4
SYSTEM ANALYSIS

4.1 EXISTING SYSTEM:

- A content-based food recommendation system was introduced, which suggests recipes based on the user's previously stated preferences. The selected recipes are broken down into ingredients, and these ingredients are scored using the user's stored preferences. Recipes with overlapping ingredients are then recommended. However, this method does not account for nutritional value or dietary balance, and it may generate repetitive suggestions if the user's preferences remain unchanged over time.
- Another approach, designed for Android platforms, utilizes tags and latent factors to deliver personalized recipe recommendations. It relies on user-assigned ratings and tags, applying matrix factorization and latent feature vectors to generate suggestions. The inclusion of tags enhances prediction accuracy by better aligning with user preferences. Still, this method also overlooks the nutritional needs of users and does not tailor recommendations to maintain a balanced diet.
- In addition, disease prediction was conducted using symptom-based inputs.

4.1.1 DISADVANTAGES OF EXISTING SYSTEM:

Diet suggestion systems like the ones listed above are focused on certain ailments or diet plans that are out of balance. When it comes to food recommendations for specific ailments, the systems do so without understanding the severity of the disease, which might vary in different circumstances and have serious consequences for individuals. Similarly, when it comes to food recommendations for a balanced diet, nutrition elements are often overlooked, despite their importance in recommending foods and maintaining a healthy diet.

Disease can be caused by lack of vitamin also which is not discussed in existing systems

4.2 PROPOSED SYSTEM:

The System works in a Machine Learning Environment, we use multiple machine learning algorithms to check accuracy of vitamin deficiency and predict disease based on vitamin deficiency and recommend food along with disease prediction and best model is used for prediction in flask web application. When user enters vitamin values algorithm will predict deficiency is vitamin and recommend food.

.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM:

- Automates the identification of vitamin deficiencies and recommends appropriate foods accordingly.
- Previous datasets are used to training and testing.
- Accuracy of model is improved compare to existing methods.
- Predicts disease based on vitamin deficiency.

Problem Type, Statement and Goals:

The increasing rate of fast-food consumption has led to the widespread intake of unhealthy food, resulting in numerous health issues such as obesity, diabetes, and elevated blood pressure. Consequently, maintaining a balanced and nutritious diet has become essential. However, in today's fast-paced lifestyle, not everyone has the time or financial resources to consult a personal dietitian or nutritionist for customized health and dietary guidance. This report addresses the issue of unhealthy eating habits and proposes a practical solution aimed at promoting a healthier lifestyle through personalized dietary recommendations.

CHAPTER- 5
SYSTEM DESIGN

5.1 INTRODUCTION

System Design Introduction:

The System Design Document outlines the system requirements, operating environment, architecture of the system and its subsystems, file and database structures, input and output formats, user interfaces, detailed design specifications, processing logic, and external system interfaces.

5.2 MODULES

5.2.1 DATASET:

In this project, we are utilizing vitamin, food recommendation, and disease detection datasets. The vitamin dataset is prepared based on the minimum and maximum values obtained from test results. The features include the minimum and maximum levels of vitamins A, B, C, D, E, and K, while the labels indicate whether there is a deficiency or not.

5.2.2 PRE-PROCESSING:

Features are extracted from the dataset and stored in the X_train variable, while the corresponding labels are stored in the y_train variable. The data is then preprocessed using the StandardScaler function, resulting in scaled features and corresponding labels for further model training

.5.2.3 METHODOLOGY:

As seen from the above figure, we can see how the data is divided into different sets and then trained for different models.

- The dataset was first divided into training set (80%) and pre-training set(20%).
- The pre-training set was divided into pre-train(80%) and pre-test(20%)
- Now, the training set is further is divided into train(80%) and validation set(20%). This train set is again divided into train(80%) and test set(20%). So, now I have train validation and test sets separate which are nonoverlapping.

• The pretrain set was used to find the best models for the given dataset. I took best 4 models using pretest set. Their performance was compared based on their mean absolute errors.

• Once the best 4 models were obtained, hyperparameters for these models were tuned and the best parameter was selected.

MODULES:

Data collection:

- In this project se are using vitamin dataset and food recommendation dataset which is prepared based on min and max vitamin values from the test results and features are min and max values of vitamin a, b, c,d,e,k values and labels are deficiency and non-deficiency.
- Based on the vitamin deficiency food data set is prepared with various combinations. In this feature are vitamin deficient values and labels are type of food.
- Based on vitamin deficiency disease prediction dataset is prepared. It has features as vitamin deficient values and labels as disease

Data preprocessing:

Features are extracted from the dataset and stored in the x_train variable, while the labels are stored in the y_train variable. The data is then preprocessed using the StandardScaler function, generating scaled features and labels.

Testing training:

- In this stage data is sent to testing and training function and divided in to four parts x test train, and y test train. Train variables are used for passing to algorithm whereas test are used for calculating accuracy of the algorithm.

Initializing Multiple Algorithms and training with Logistic regression:

- At this stage, machine learning algorithms are initialized and trained using the input features and labels. The trained model is then saved as a pickle file for future predictions.
- The dataset is trained with multiple algorithms, their accuracies are evaluated, and the best-performing model is selected for prediction

Predict data:

- In this stage new data is taken as input and trained models are loaded using pickle and then values are preprocessed and passed to predict function to find out result which is showed on web application.

5.3 SYSTEM ARCHITECTURE

A system architecture is a conceptual model that defines the structure, behavior, and various perspectives of a system. It provides a formal and organized representation that supports understanding and reasoning about the system's components and their interactions.

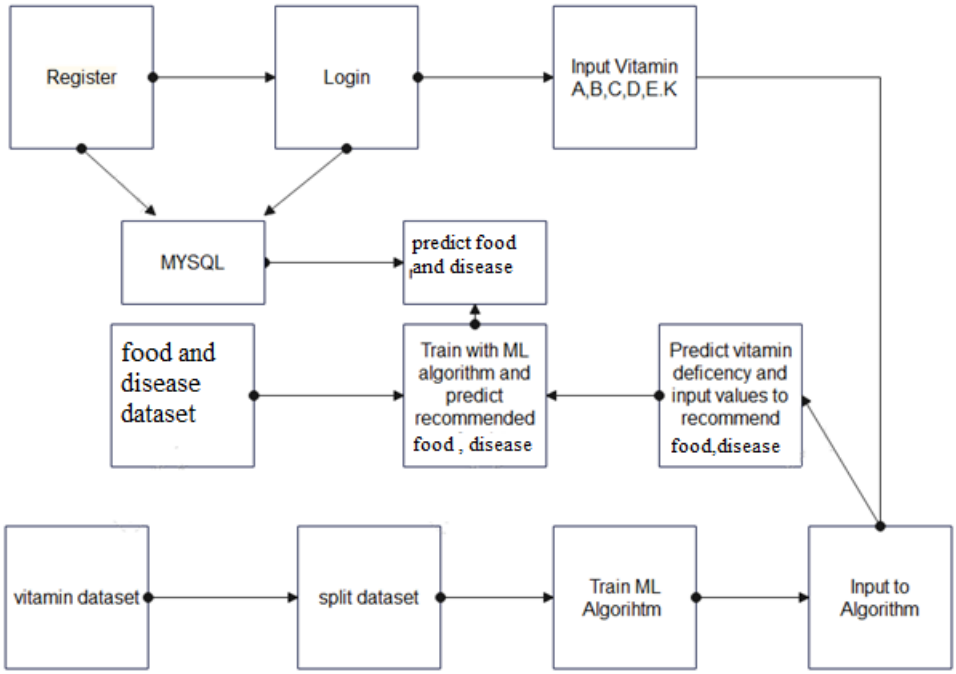


Figure 5. 1 System Architecture

3-Tier Architecture:

The three-tier software architecture, introduced in the 1990s, evolved to address the limitations of the two-tier model. It adds a middle tier—also known as the application or logic tier—between the user interface (client) and the data management (server) layers. This middle tier handles process management, including business logic and rule execution, and supports a significantly higher number of users (hundreds, compared to about 100 in the two-tier model) through features like queuing, application execution, and database staging.

This architecture is ideal for distributed client/server systems, offering improved performance, scalability, flexibility, maintainability, and reusability. It also simplifies the user experience by

hiding the complexity of distributed processing, making it a preferred choice for web-based applications and net-centric information systems.

Advantages of Three-Tier:

- Separates functionality from presentation.
- Clear separation – better understanding.
- Changes limited to well define components.
- Can be running on WWW.
- Effective network performance.

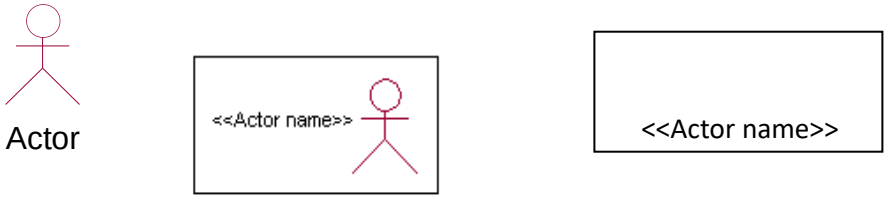
5.4 UML DAIGRAMS

Global Use Case Diagrams:

Identification of actors:

Actor: Actor represents the role a user plays with respect to the system. An actor interacts with, but has no control over the use cases.

Graphical representation:



An actor is someone or something that:

Interacts with or uses the system.

- Provides input to and receives information from the system.
- Is external to the system and has no control over the use cases.

Actors are discovered by examining:

- Who directly uses the system?
- Who is responsible for maintaining the system?
- External hardware used by the system.
- Other systems that need to interact with the system.

Questions to identify actors:

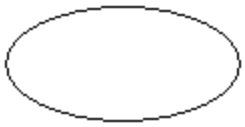
- Who is using the system? Or, who is affected by the system? Or, which groups need help from the system to perform a task?
- Who affects the system? What types of users interact with the system to carry out its core and supplementary operations, such as administrative tasks?
- Are there any external devices or systems that interface with this system to complete specific operations?
- What challenges or issues does this application address, and who benefits from its solutions?
- Lastly, how do users interact with the system in practice? What are the typical actions or tasks they perform using it?
- The actors identified in this system are:

a. System Administrator
b. Customer
c. Customer Care

Identification of use cases:

Use case: A use case can be described as a specific way of using the system from a user's (actor's) perspective.

Graphical representation:



A more detailed description might characterize a use case as:

- Pattern of behavior the system exhibits
- A series of interconnected actions or operations carried out by a user (actor) in collaboration with the system.
- Delivering something of value to the actor

Use cases provide a means to:

- capture system requirements
- communicate with the end users and domain experts
- test the system

Use cases are best discovered by examining the actors and defining what the actor will be able to do with the system.

Guide lines for identifying use cases:

- Identify the actions and responsibilities associated with each actor. The use cases should outline a sequence of events that result in a clearly defined goal.
- Assign descriptive and relevant names to each use case.
- Provide concise explanations of the use cases using terminology that is familiar to the user. This helps ensure clarity and minimizes potential confusion

Questions to identify use cases:

- What are the tasks of each actor?
- Will any actor create, store, change, remove or read information in the system?
- What use case will store, change, remove or read this information?
- Will any actor need to inform the system about sudden external changes?
- Does any actor need to inform about certain occurrences in the system?

- What usecases will support and maintains the system?

1.2 Flow of Events

A flow of events outlines a series of interactions or steps carried out by the system. These events are usually described in detail, focusing on *what* the system must accomplish rather than *how* it achieves those tasks. Typically, these flows are documented separately using any preferred text editor and then linked to a specific use case via the Files tab in the corresponding model element.

A flow of events should include:

- When and how the use case starts and ends
- Use case/actor interactions
- Data needed by the use case
- Normal sequence of events for the use case
- Alternate or exceptional flows

5.4.1 CONSTRUCTION OF USE CASE DIAGRAMS:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

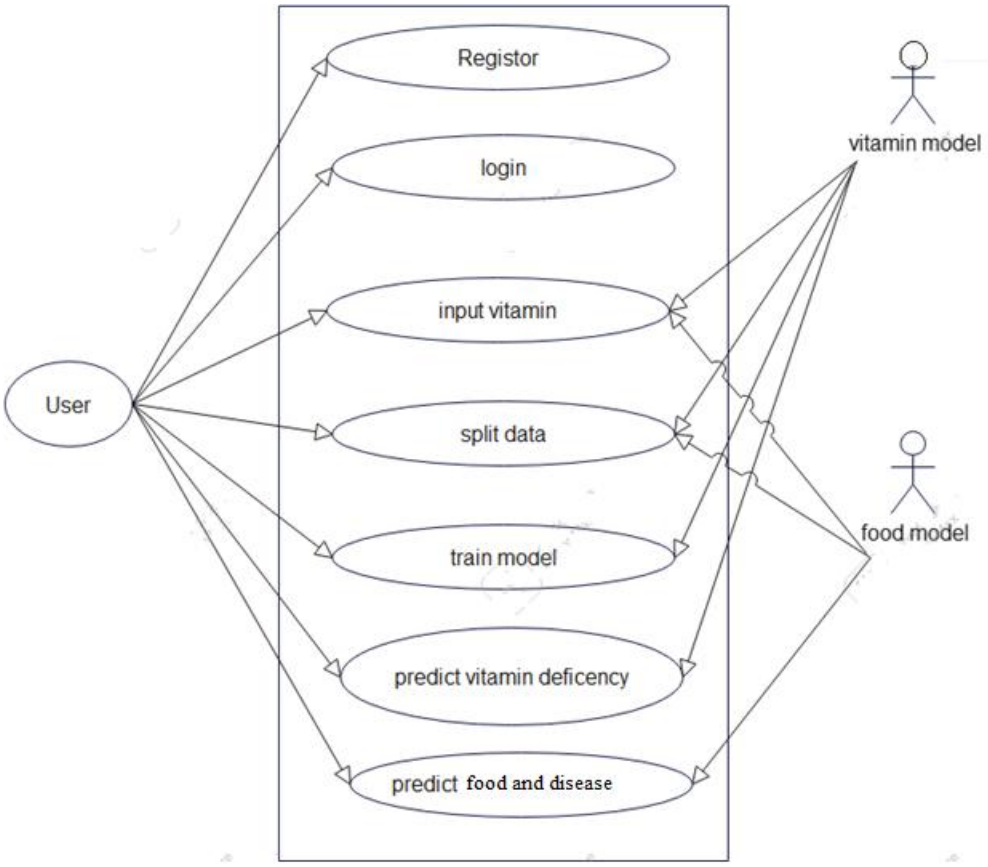


Figure 5. 2 Use Case Diagram

5.4.2 SEQUENCE DIAGRAMS:

A sequence diagram in Unified Modeling Language (UML) is a type of interaction diagram that illustrates how objects or components interact over time. It visually represents the order in which messages are exchanged between different parts of the system. Sequence diagrams are also referred to as event scenarios, interaction diagrams, or timing diagrams.

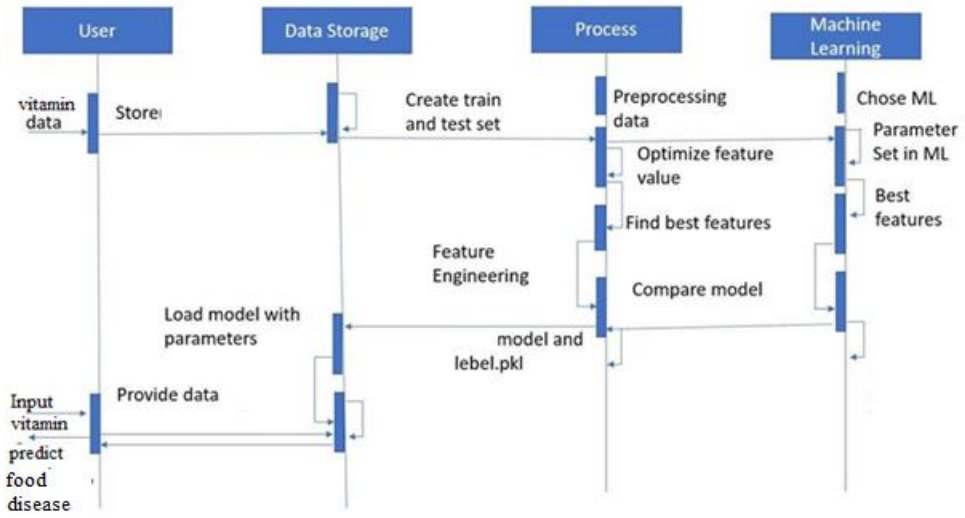


Figure 5. 3 Sequence diagram

5.4.3 CLASS DIAGRAM:

In software engineering, a class diagram is a type of static structure diagram in the Unified Modeling Language (UML) that outlines the system's architecture. It represents the system's classes, including their attributes, methods, and the relationships between them. This diagram helps in understanding how data and functionality are organized within the system.

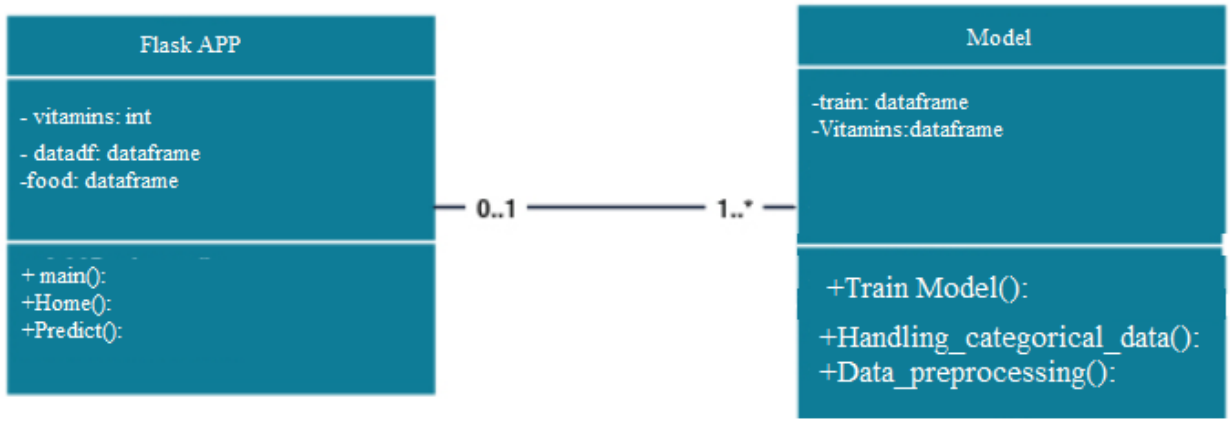


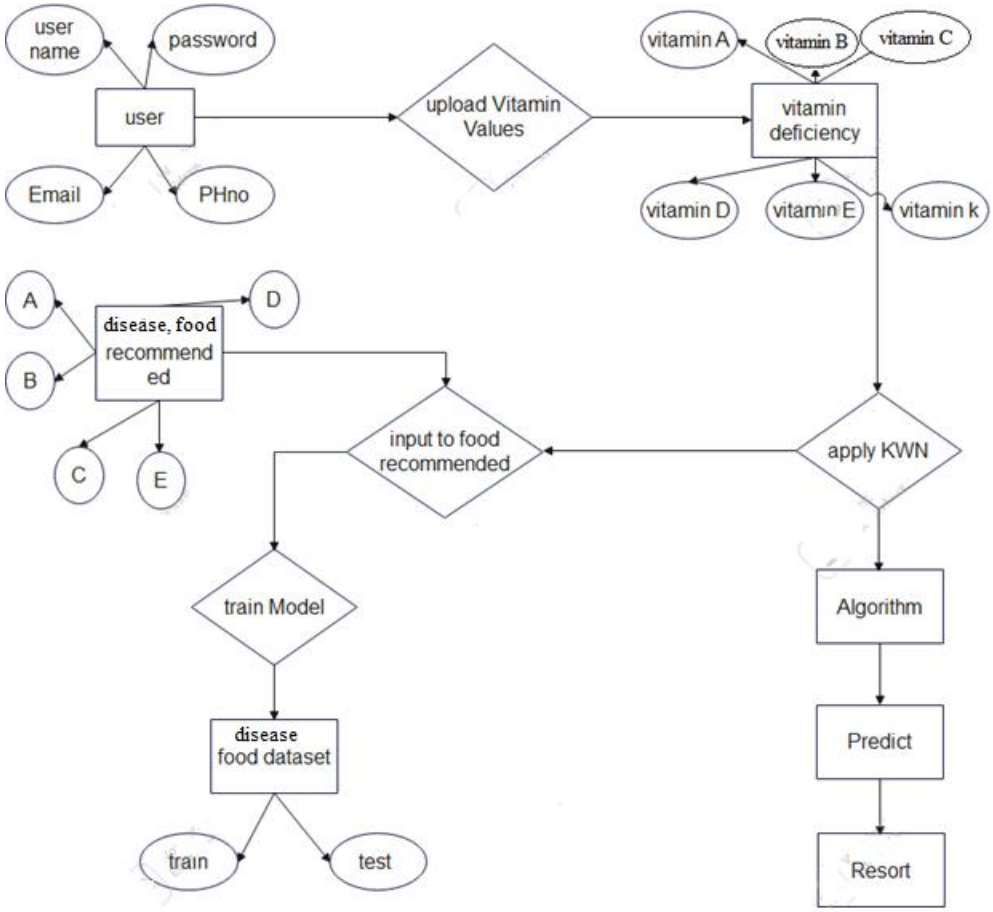
Figure 5. 4 Class Diagram

5.4.4 ACTIVITY DIAGRAM:

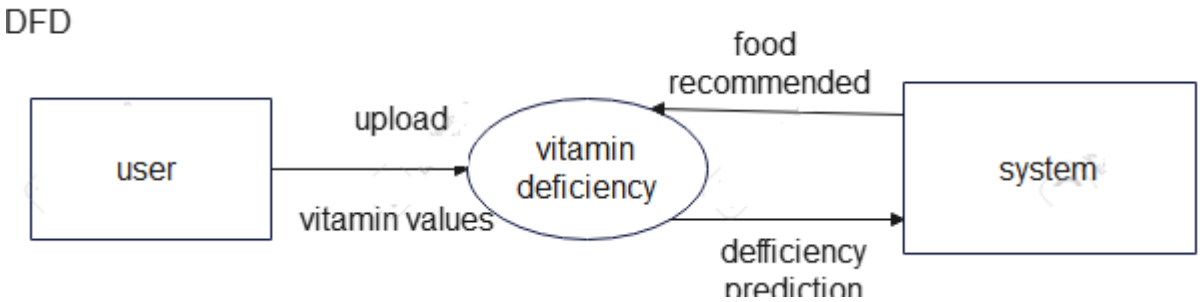
Activity diagrams visually represent workflows composed of a series of actions or steps, incorporating decisions, loops, and parallel processes. In Unified Modeling Language (UML), these diagrams are useful for modeling the business processes and operational workflows within a system. They provide a clear view of how control flows throughout the various activities involved.

Figure 5. 5 Activity Diagram

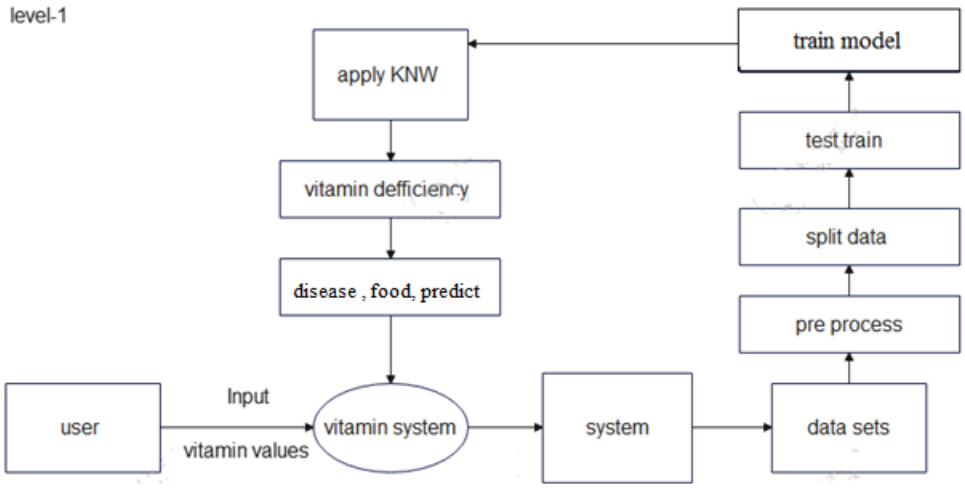
5.4.4 ER DIAGRAM



DFD LEVEL 0:



DFD LEVEL 1:



CHAPTER-6

6.1 SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS:

- SYSTEM : INTEL(R) CORE(TM) I3-7020U CPU @ 2.30GHZ
- HARD DISK : 1 TB.
- INPUT DEVICES : KEYBOARD, MOUSE
- RAM : 4 GB.

SOFTWARE REQUIREMENTS:

- OPERATING SYSTEM : WINDOWS XP/7/10.
- CODING LANGUAGE : PYTHON
- TOOL : ANACONDA
- INTERFACE : JUPITER NOTEBOOK

CHAPTER-7
SYSTEM IMPLEMENTATION

To conduct studies and analyses of an operational and technological nature, and To promote the exchange and development of methods and tools for operational analysis as applied to defense problems.

7.1 INPUT AND OUTPUT DESIGNS

7.1.1 LOGICAL DESIGN

The logical design of a system pertains to an abstract representation of the data flows, inputs and outputs of the system. This is often conducted via modeling, using an over-abstract (and sometimes graphical) model of the actual system. In the context of systems design are included. Logical design includes ER Diagrams i.e. Entity Relationship Diagrams

7.1.2 PHYSICAL DESIGN

The physical design focuses on the real-world aspects of system operation, specifically how data is entered, validated or authenticated, processed, and ultimately presented as output. It outlines the technical details of input and output methods. During this stage, key decisions are made regarding how the system will function in practice, including hardware specifications, interface layouts, and the flow of data through the system.

1. Input requirement,
2. Output requirements,
3. Storage requirements,
4. Processing Requirements,
5. System control and backup or recovery.

In other words, the physical aspect of systems design is typically divided into three main components or sub-tasks:

1. ‘User Interface Design
2. Data Design
3. Process Design

User Interface Design focuses on how users input data into the system and how the system delivers information back to them. Data Design deals with the internal structure and storage of information within the system. Process Design emphasizes the movement of data across the system, detailing how it is validated, secured, or transformed as it enters, travels through, and exits the system. Once the design phase is completed, documentation covering these three sub-tasks is created and prepared for the next development stage.

In this context, physical design doesn’t refer to the actual physical layout of a system. To illustrate, a personal computer's physical design includes data input through the keyboard, processing via the CPU, and output through devices like the monitor or printer. It does not involve the arrangement of the hardware itself, such as the CPU casing, motherboard, hard drive, graphics card, or USB ports. Instead, it includes a detailed structure of the user interface, product database, processing units, and control systems. The hardware and software specifications are prepared for the proposed system

7.2 INPUT & OUTPUT REPRESENTATION

7.2.1 INPUT DESIGN

Input design serves as the bridge between the user and the information system. It involves defining the specifications and procedures needed for preparing data before it enters the system for processing. This step ensures that transaction data is converted into a form that the system can use. Data entry can be done either through direct manual input by users or by using automated methods such as scanning printed or written documents.

The goal of input design is to minimize the amount of data required, reduce input errors, eliminate unnecessary steps, and keep the process efficient and user-friendly. It is also important that input mechanisms maintain data security, ensure ease of use, and protect user privacy. Key considerations during input design include:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

7.2.2 OBJECTIVES

Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

User-friendly data entry screens are developed to efficiently handle large volumes of input. The primary objective of input design is to simplify data entry while minimizing the chances of errors. These screens are designed to support various data manipulation tasks and also offer options for viewing records.

During the input process, the system performs validity checks to ensure the accuracy and consistency of the entered data. Users interact with the system through these screens, and helpful messages are displayed when necessary to guide them through the process. Overall, the goal of input design is to develop a layout that is intuitive and easy for users to navigate.

7.2.3 Output Design

An effective output is one that fulfills the needs of the end user and presents information in a clear and understandable manner. In any system, the outcome of data processing must be communicated either to users or to other systems through well-designed outputs. Output design involves deciding

how information should be displayed for immediate viewing as well as how it should be presented in printed (hard copy) form.

Outputs are the most direct form of communication between the system and its users. Well-structured and meaningful outputs enhance the overall user experience and support better decision-making. A thoughtfully designed output format contributes significantly to the system's usability and effectiveness

a. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.
b. Select methods for presenting information.
c. Create document, report, or other formats that contain information produced by the system.

Code

```
import os
import MySQLdb
from flask import Flask, session, url_for, redirect, render_template,
request, abort, flash
from database import db_connect
from database import
db_connect,vit_act,view_vit,user_reg,user_loginact,user_viewimages,vit
_info,get_recomendations,getdisease_recomendations
from database import db_connect
from werkzeug.utils import secure_filename



app = Flask(__name__)
app.secret_key = os.urandom(24)
PEOPLE_FOLDER = os.path.join('static', 'images')
app.config['UPLOAD_FOLDER'] = PEOPLE_FOLDER

@app.route("/")
def FUN_root():
    return render_template("index.html")

@app.route("/index.html")
def logout():
    return render_template("index.html")

# @app.route("/upload.html")
# def upload():
#     return render_template("upload.html")

@app.route("/register.html")
def reg():
    return render_template("register.html")

@app.route("/login.html")
def login():
    return render_template("login.html")

# @app.route("/upload.html")
# def up():
#     username = session['username']
#     data = view_vit(username)
#     return render_template("upload.html" , data = data)

@app.route("/viewdata.html")
def up1():
    return render_template("viewdata.html")
```

```
# ---------------------------------------------register-----------------
--------------------------------------
@app.route("/regact", methods = ['GET','POST'])
def registeract():
   if request.method == 'POST':
      id="0"
      status =
user_reg(id,request.form['username'],request.form['password'],request.
form['email'],request.form['mobile'],request.form['address'])
      if status == 1:
       return render_template("login.html",m1="sucess")
      else:
       return render_template("register.html",m1="failed")


@app.route("/Vitaminact", methods = ['GET','POST'])
def Vitaminact1():
   if request.method == 'POST':
      username=session['username']
      status =
vit_act(username,request.form['vita'],request.form['vitb'],request.for
m['vitc'],request.form['vitd'],request.form['vite'],request.form['vite
k'])
      #data = view_vit(username)
      print("dfdfdfdfdfdfdfdfdfdfdfdfdfdfdfdfdfdfdfdfd")
      print(status)
      if status[0]==1:
          vara="Deficient"
      else:
          vara="In Range"
      if status[1]==1:
          varb="Deficient"
      else:
          varb="In Range"
      if status[2]==1:
          varc="Deficient"
      else:
          varc="In Range"
      if status[3]==1:
          vard="Deficient"
      else:
          vard="In Range"
      if status[4]==1:
          vare="Deficient"
      else:
          vare="In Range"
      if status[5]==1:
          vark="Deficient"
      else:
          vark="In Range"
      reco=get_recomendations(status[6])
      recom=getdisease_recomendations(status[7])
```

```
      print(status[6])
      print(recom[0])
      if status[6]==1:
         full_filename = os.path.join(app.config['UPLOAD_FOLDER'],
'fd.jpg')
      if status[6]==2:
         full_filename = os.path.join(app.config['UPLOAD_FOLDER'],
'fcvmd.jpg')
      if status[6]==3:
         full_filename = os.path.join(app.config['UPLOAD_FOLDER'],
'cereals.jpg')
      if status[6]==4:
         full_filename = os.path.join(app.config['UPLOAD_FOLDER'],
'dryfruits.jpg')
      if status[6]==5:
         full_filename = os.path.join(app.config['UPLOAD_FOLDER'],
'fruits.jpg')
      if status[6]==6:
         full_filename = os.path.join(app.config['UPLOAD_FOLDER'],
'vegetables.jpg')


      if status:
       return
render_template("upload.html",m1="sucess",first=vara,second=varb,third
=varc,fourth=vard,fifth=vare,sixth=vark,seventh=reco[0],eight=recom[0]
,user_image = full_filename)
      else:
       return render_template("userhome.html",m1="failed")
#---------------------------------------------Login--------------------
---------------------------------
@app.route("/loginact", methods=['GET', 'POST'])
def useract():
    if request.method == 'POST':
        status = user_loginact(request.form['username'],
request.form['password'])
        print(status)
        if status == 1:
            session['username'] = request.form['username']
            return render_template("userhome.html", m1="sucess")
        else:
            return render_template("login.html", m1="Login Failed")
#---------------------------------------------Upload Image--------------
--------------------
# @app.route("/upload", methods = ['GET','POST'])
# def upload():
#    if request.method == 'POST':
#       id="0"
#       status =
user_upload(id,request.form['name'],request.form['image'])
#       if status == 1:
#        return render_template("upload.html",m1="sucess")
```

```
#       else:
#        return render_template("upload.html",m2="failed")
#-------------------------------------View Images--------------------
---------------------
@app.route("/viewimage.html")
def viewimages():
    data = user_viewimages(session['username'])

    print(data)
    return render_template("viewimage.html",user = data)



@app.route("/track")
def track():
    username = request.args.get('username')
    vita = request.args.get('vita')
    vitb = request.args.get('vitb')
    vitc = request.args.get('vitc')
    vitd = request.args.get('vit')
    vite = request.args.get('vite')

    data = vit_info(username,vita,vitb,vitc,vitd,vite)
    print("ddddddddddddddddddddddddddddd")
    print(data)
    print("ddddddddddddddddddddddddddddd")
    print(data)
    return render_template("viewimage.html",m1="sucess",data=data)

#---------------------------------------Track------------------------
----------------------
# @app.route("/track")
# def track():
#     name = request.args.get('name')
#     iname = request.args.get('iname')

#     data = image_info(iname)
#     print("ddddddddddddddddddddddddddddd")
#     print(data)
#     data = v_image(data)
#     print("ddddddddddddddddddddddddddddd")
#     print(data)
#     return render_template("viewdata.html",m1="sucess",users=data)


# ----------------------------------------------Update Item-----------
-------------------------------
if __name__ == "__main__":
    app.run(debug=True, host='127.0.0.1',
port=5000,use_reloader=False)
```

CHAPTER-8
SYSTEM TESTING

8.1 INTRODUCTION:

Testing is the debugging program is one of the most critical aspects of the computer programming triggers, without programming that works, the system would never produce an output of which it was designed. Testing is best performed when user development is asked to assist in identifying all errors and bugs. The sample data are used for testing. It is not quantity but quality of the data used the matters of testing. Testing is aimed at ensuring that the system was accurately an efficiently before live operation commands.

Testing objectives:

The main objective of testing is to uncover a host of errors, systematically and with minimum effort and time. Stating formally, we can say, testing is a process of executing a program with intent of finding an error.

1 A successful test is one that uncovers an as yet undiscovered error.
2 A good test case is one that has probability of finding an error, if it exists.
3 The test is inadequate to detect possibly present errors.
4 The software more or less confirms to the quality and reliable standards.

8.2 LEVELS OF TESTING

Code testing:

This examines the logic of the program. For example, the logic for updating various sample data and with the sample files and directories were tested and verified.

Specification Testing:

Executing this specification starting what the program should do and how it should performed under various conditions. Test cases for various situation and combination of conditions in all the modules are tested.

Unit testing:

In the unit testing we test each module individually and integrate with the overall system. Unit testing focuses verification efforts on the smallest unit of software design in the module. This is also known as module testing. The module of the system is tested separately. This testing is carried out during programming stage itself. In the testing step each module is found to work satisfactorily as regard to expected output from the module. There are some validation checks for fields also. For example the validation check is done for varying the user input given by the user which validity of the data entered. It is very easy to find error debut the system.

Each Module can be tested using the following two Strategies:

1 Black Box Testing
2 White Box Testing

8.2.1 BLACK BOX TESTING

What is Black Box Testing?

Black Box Testing is a software testing approach where the internal workings, structure, or design of the software are not known to the tester. Instead, the testing process focuses solely on evaluating the functionality of the software by examining its inputs and expected outputs based on requirements and specifications.

In this method, testers assess how the system behaves under various conditions, ensuring that it produces the correct results for given inputs. The emphasis is on verifying whether the application meets user expectations and performs as intended, without any need to understand how the system is implemented internally.

The "black box" in black box testing refers to any software system whose internal workings are hidden from the tester. This could be anything—from an operating system like Windows, to a search engine like Google, a database system like Oracle, or even a custom-built application. The key idea is that testing is performed by interacting with the system through its inputs and analyzing the outputs, without any knowledge of the underlying code or logic. Here are the generic steps followed to carry out any type of Black Box Testing.

- Begin by analyzing the system's requirements and specifications to understand its expected behavior.
- Select appropriate test inputs:
  - Use **valid inputs** to verify that the system handles them correctly (positive testing).
  - Use **invalid inputs** to confirm that the system appropriately detects and handles errors (negative testing).
- Define the expected results for each set of inputs.
- Create test cases based on these inputs and expectations.
- Run the test cases by providing inputs to the system under test (SUT).
- Compare the actual output received from the system with the expected results.
- If any discrepancies or issues are found, report them as defects.
- Once the defects are resolved, re-run the tests to ensure the issues are properly fixed.

Types of Black Box Testing

There are many types of Black Box Testing but following are the prominent ones -

- **Functional Testing**:This focuses on verifying that the software performs according to its functional requirements. Testers examine whether specific features work as intended without knowing the internal code structure.
- **Non-Functional Testing**:Instead of checking specific functionalities, this testing evaluates aspects like system performance, scalability, reliability, and user experience. It ensures the application meets broader system requirements.
- **Regression Testing:**Conducted after code changes such as bug fixes or system upgrades, this type ensures that the new changes haven't negatively impacted the existing functionality of the application. It helps maintain system stability over time

8.2.2 WHITE BOX TESTING

White Box Testing refers to the method of testing a software application by examining its internal code structure and logic. This technique mainly aims to enhance **security**, ensure the correct **flow of data**, and improve **design and functionality**.

Also known as **clear box**, **glass box**, or **structural testing**, it involves having full visibility into the internal workings of the software — hence the term *white box*, symbolizing transparency. In contrast, **Black Box Testing** focuses on evaluating the software from the user's point of view without any knowledge of the internal processes.

WHAT DO YOU VERIFY IN WHITE BOX TESTING?

White box testing involves the testing of the software code for the following:

- Internal security holes
- Broken or poorly structured paths in the coding processes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object and function on an individual basis

White box testing can be applied during various stages of the software development lifecycle, including **unit**, **integration**, and **system** levels. A key objective of this testing method is to ensure that the application's logic and workflows operate correctly. It involves providing a set of known inputs and checking whether the resulting outputs align with the expected outcomes. If the actual output deviates from what's anticipated, it indicates the presence of a **defect** or **bug** in the code.

HOW DO YOU PERFORM WHITE BOX TESTING?

To give you a simplified explanation of white box testing, we have divided it into **two basic steps**. This is what testers do when testing an application using the white box testing technique:

STEP 1) UNDERSTAND THE SOURCE CODE

One of the initial steps in white box testing is for the tester to thoroughly understand the application's source code. Since this type of testing focuses on the internal logic and structure of the software, it requires the tester to be well-versed in the programming languages used in its development. Additionally, a strong understanding of secure coding standards is crucial, as identifying and mitigating security vulnerabilities is often a top priority. The tester should be capable of detecting potential security flaws and protecting the application from possible threats, including those caused by intentional or unintentional injection of malicious code by users.

Step 2) CREATE TEST CASES AND EXECUTE

The next essential phase in white box testing focuses on examining the source code to ensure correct logic and structure. This is often achieved by writing additional test scripts specifically designed to verify various components or sequences within the application. These targeted tests are typically created by someone with a deep understanding of the code, often the developer themselves. In addition to this, other techniques such as manual inspection, trial-and-error testing, and specialized testing tools can also be employed, which we'll explore in more detail later in this document.

Unit testing:

Sl # Test Case :	UTC1
Name of Test:	Data Set Feature Label Extraction
Items being tested:	Features and Labels are extracted or not
Sample Input:	Csv files vitamin dataset and food recommendation dataset
Expected output:	Features copied to x labels to y
Actual output:	Data with features and labels are displayed.
Remarks:	**Pass.**

Sl # Test Case :	UTC2
Name of Test:	Register login
Items being tested:	User registration details stored in Database and login success or Fail
Sample Input:	Username, password, email, phone number, address
Expected output:	Registration successful, login success
Actual output:	Details stored in database and verification is success for

	correct details
Remarks:	pass

Integration Testing:

Integration testing is a software testing phase where multiple individual modules are combined and evaluated as a group. The main objective is to identify defects that arise from the interaction between these integrated units. To support this process, tools like test drivers and test stubs are often used. This testing stage takes place after unit testing and before validation testing. Integration testing ensures that interconnected components work together as intended and can be carried out using approaches such as bottom-up integration and top-down integration.

Bottom-up Integration

This testing begins with unit testing, followed by tests of progressively higherlevel combinations of units called modules or builds.

Top-down Integration

In this testing, the highest-level modules are tested first and progressively, lower-level modules are tested thereafter.

In a thorough software development setup, bottom-up integration testing is typically performed initially, followed by top-down testing. The testing process is finalized through multiple rounds of evaluation on the complete system, ideally using real-world scenarios that closely simulate actual user environments.

Table 6.5shows the test cases for integration testing and their results

Sl # Test Case :	ITC1
Name of Test:	Input vitamin values
Item being tested:	Vitamin difficiency is predicted or not
Sample Input:	Vitamin, A, b, c, d, e, k values
Expected output:	Which vitamin is deficient which one is normal.
Actual output:	Vitamin name with deficient values are displayed.
Remarks:	Pass.

Sl # Test Case :	ITC2
Name of Test:	Predict food recommendation
Item being tested:	Prediction is success or not for vitamin deficient values
Sample Input:	Vitamin predicted values
Expected output:	Food recommendation with images
Actual output:	Food recommendation images
Remarks:	Pass.

System testing:

System testing is performed on a fully integrated software or hardware system to assess whether it meets the specified requirements. As a type of black-box testing, it does not involve examining the internal structure or logic of the system. The importance of system testing lies in several key areas:

- It marks the first phase in the Software Development Life Cycle (SDLC) where the system is evaluated in its entirety.
- It ensures the application aligns with both functional and technical specifications.
- Testing is carried out in an environment that closely resembles the actual production setting.

- This phase allows for comprehensive verification and validation of both business logic and the overall system architecture.

System Testing is shown in below tables

Sl # Test Case : -	STC-1
Name of Test: -	System testing in various versions of OS
Item being tested: -	OS compatibility.
Sample Input: -	Execute the program in windows XP/ Windows-7/8
Expected output: -	Performance is better in windows-7
Actual output: -	Same as expected output, performance is better inwindows-7
Remarks: -	Pass

CHAPTER-9
OUTPUT SCREENS

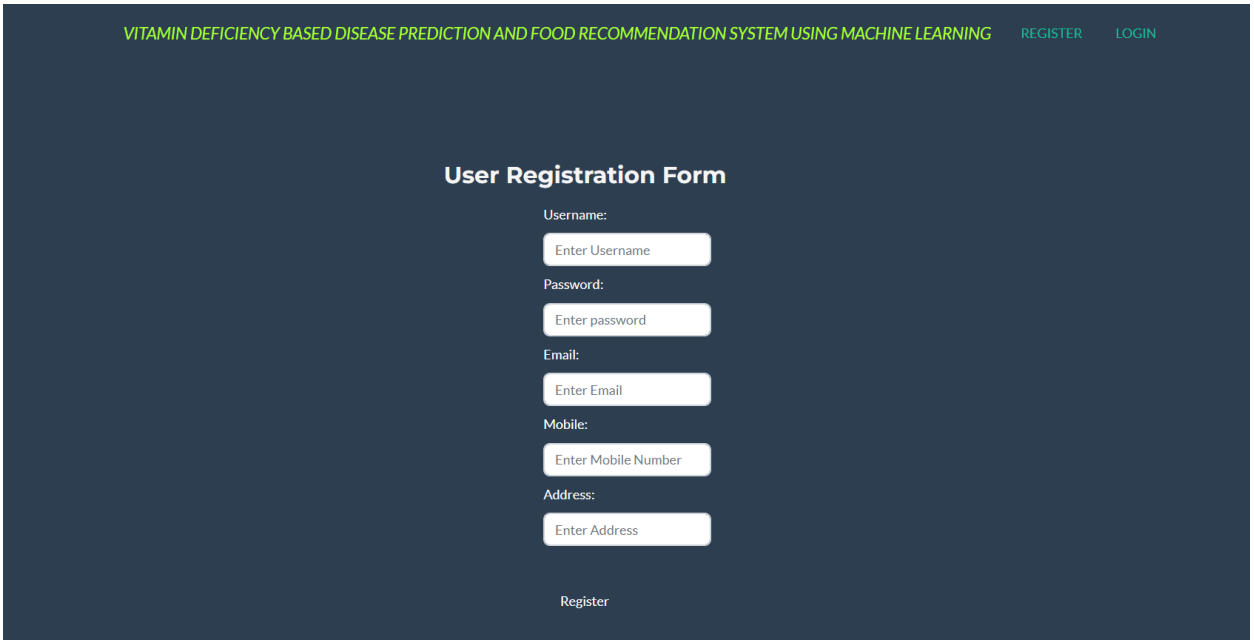
9.1 MAIN PAGE

9.2 Datasets

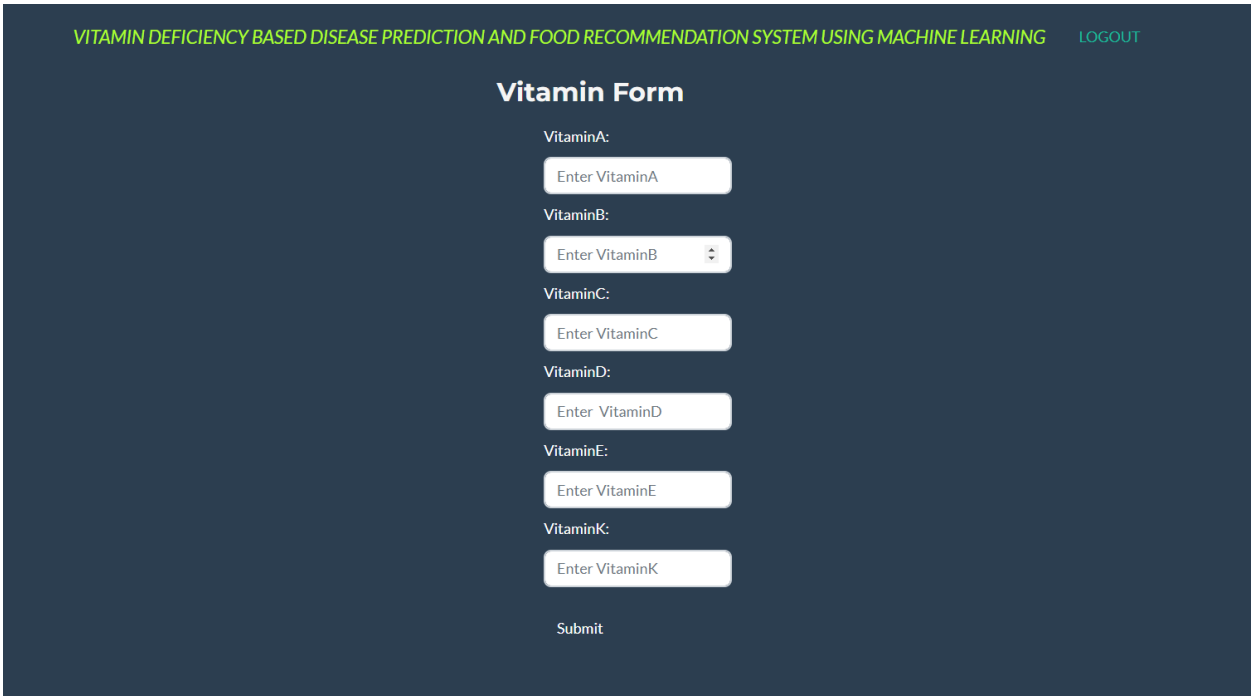
vitaminA	DificencyA	vitaminC	DificencyC	vitaminD	DificencyD	vitaminK	Dificencyk	vitaminB	DificencyB	vitaminE	DificencyE	Dif.Vitamii	Dif.Vitamii	Dif.Vitamii	Dif.Vitamii	Dif.Vitamii	Dif.VitaminK	Recommend
0.1	1	0.1	1	20	0	0.2	1	10	1	1.1	1	1	0	0	0	0	0	Fruits/Dryfruits
0.2	1	0.2	1	21	0	0.3	1	20	1	1.2	1	1	1	0	0	0	0	Fruits/cereals/vegetables/millets/Dryfruits
0.3	1	0.3	0	22	0	0.4	1	30	1	1.3	1	1	1	1	0	0	0	Fruits/cereals/vegetables/millets/Dryfruits
0.4	1	0.4	0	23	0	0.5	1	40	1	1.4	1	1	1	1	1	0	0	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
0.5	1	0.5	0	24	0	0.6	1	50	1	1.5	1	1	1	1	1	1	0	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
0.6	1	0.6	0	25	0	0.7	1	60	1	1.6	1	0	1	0	0	0	0	cereals/vegetables/millets/Dryfruits
0.7	1	0.1	1	26	0	0.8	1	70	1	1.7	1	0	1	1	0	0	0	Fruits/cereals/vegetables/millets/Dryfruits
0.8	1	0.2	1	27	0	0.9	1	80	1	1.8	1	0	1	1	1	0	0	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
0.9	1	0.3	0	28	0	1	1	90	1	1.9	1	0	1	1	1	1	0	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
1	1	0.4	0	29	0	1.1	1	100	1	2	1	0	1	1	1	1	1	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
1.1	1	0.5	0	30	0	1.2	1	110	1	2.1	1	0	0	1	0	0	0	Fruits/Dryfruits
1.2	1	0.6	0	31	0	1.3	1	120	1	2.2	1	0	0	1	1	0	0	Fruits/cheese/curd/milk/meat/Dryfruits
1.3	1	0.1	1	32	0	1.4	1	130	1	2.3	1	0	0	1	1	1	0	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
1.4	1	0.2	1	33	0	1.5	1	140	1	2.4	1	0	0	1	1	1	1	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
1.5	1	0.3	0	34	0	1.6	1	150	1	2.5	1	0	0	0	1	0	0	cheese/curd/milk/meat/Dryfruits
1.6	1	0.4	0	35	0	1.7	1	160	1	2.6	1	0	0	0	1	1	0	cereals/vegetables/millets/Dryfruits/cheese/curd/m
1.7	1	0.5	0	36	0	1.8	1	170	0	2.7	1	0	0	0	1	1	1	Fruits/cereals/vegetables/millets/Dryfruits/cheese/c
1.8	1	0.6	0	37	0	1.9	1	180	0	2.8	1	0	0	0	0	1	0	cereals/vegetables/millets/Dryfruits
1.9	1	0.1	1	38	0	2	1	190	0	2.7	1	0	0	0	0	1	1	cereals/vegetables/millets/Dryfruits
2	1	0.2	1	39	0	2.1	1	200	0	2.9	1	0	0	0	0	0	1	cereals/vegetables/millets/Dryfruits
2.1	0	0.3	0	40	0	2.2	1	210	0	3	1							
2.2	0	0.4	0	41	1	2.3	1	220	0	3.1	1							
2.3	0	0.5	0	42	1	2.4	1	230	0	3.2	1							
2.4	0	0.6	0	43	1	2.5	1	240	0	3.3	1							
2.5	0	0.1	1	44	1	2.6	1	250	0	3.4	1							
2.6	0	0.2	1	45	1	2.7	1	260	0	3.5	1							

DifVitaminA	DifVitaminB	DifVitamin	DifVitamin	DifVitamin	DifVitaminK	CLASS	Disease
1	0	0	0	0	0	1	Xerophthalmia
1	1	0	0	0	0	2	anemia
1	1	1	0	0	0	3	bruising, gum and dental problems, dry hair and skin, and anemia
1	1	1	1	0	0	4	Osteomalacia
1	1	1	1	1	0	5	Crohns disease
0	1	0	0	0	0	6	cystic fibrosis
0	1	1	0	0	0	7	diabetes
0	1	1	1	0	0	8	chronic kidney disease
0	1	1	1	1	0	9	Fungal infection
0	1	1	1	1	1	10	Allergy
0	0	1	0	0	0	11	GERD
0	0	1	1	0	0	12	Chronic cholestasis
0	0	1	1	1	0	13	Drug Reaction
0	0	1	1	1	1	14	Peptic ulcer diseae
0	0	0	1	0	0	15	AIDS
0	0	0	1	1	0	16	Diabetes
0	0	0	1	1	1	17	Gastroenteritis
0	0	0	0	1	0	18	Bronchial Asthma
0	0	0	0	1	1	19	Hypertension
0	0	0	0	0	1	20	Migraine
1	0	0	0	0	0	21	Cervical spondylosis
1	1	0	0	0	0	22	Paralysis (brain hemorrhage)
1	1	1	0	0	0	23	Jaundice
1	1	1	1	0	0	24	Malaria
1	1	1	1	1	0	25	Chicken pox
0	1	0	0	0	0	26	Dengue

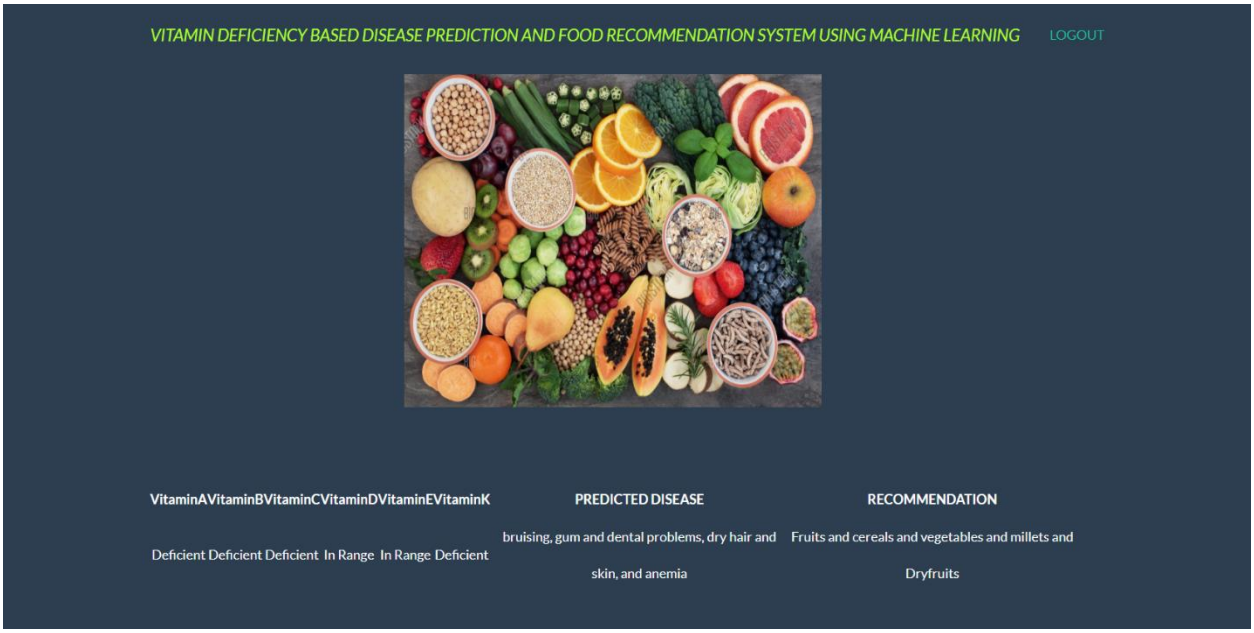
Registration page



9.3 Enter vitamin values



Predict Disease, Deficiency and food recombination



CONCLUSION

We have developed a website that recommends food items and predicts vitamin deficiencies based on user input related to vitamin levels. The system begins by preparing a dataset of food items categorized according to specific vitamin deficiencies. Using this data, the system can make predictions not only for suitable food recommendations but also for potential diseases associated with particular deficiencies. After training the model using the K-Nearest Neighbors (KNN) classifier algorithm, it identifies the closest matching food options suitable for the user's nutritional needs. This diet recommendation system helps users receive personalized healthy diet suggestions and provides alerts about possible disease risks based on predicted vitamin deficiency values.

REFERENCES

- Phanich, M., Pholkul, P., & Phimoltares, S. (2010, April). *Food recommendation system using clustering analysis for diabetic patients*. In Proceedings of the International Conference on Information Science and Applications (pp. 1–8). IEEE.
- Ge, M., Elahi, M., Fernández-Tobías, I., Ricci, F., & Massimo, D. (2015, May). *Using tags and latent factors in a food recommender system*. In Proceedings of the 5th International Conference on Digital Health (pp. 105–112). ACM.
- Freyne, J., & Berkovsky, S. (2013). *Evaluating recommender systems for supportive technologies*. In User Modeling and Adaptation for Daily Routines (pp. 195–217). Springer London.
- Khaire, P., Suvarna, R., & Chaudhary, A. (2020, January). *Virtual Dietitian: An Android based Application to Provide Diet*. International Research Journal of Engineering and Technology (IRJET), 7(1).
- Singh, S., Bait, S., Rathod, J., & Pathak, N. (2020, January). *Diabetes Prediction Using Random Forest Classifier and Intelligent Dietician*. International Research Journal of Engineering and Technology (IRJET), 7(1).